



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Εργαστηριακή Άσκηση Αρχές Γλωσσών Προγραμματισμού και Μεταφραστών

Μέλη Ομάδας

- Βασίλειος Ασημακόπουλος AM: 1084610 4ο Έτος
- Ελευθερία Βαλαχά AM: 1084602 4ο Έτος
- Παντελεήμων Καραγιάννης AM: 1084582 4ο Έτος
- Γεωργία Καραχάλιου AM: 1084628 4ο Έτος

Περιεχόμενα:

- 1. Γραμματική BNF**
- 2. Παραδοχές**
- 3. Τελικό Αρχείο FLEX**
- 4. Τελικό Αρχείο BISON**
- 5. Παραδείγματα Εκτέλεσης**
- 6. Παραδοτέα Αρχεία και Makefile**

1.Γραμματική BNF

<start> ::= <program> <start>
 | error "\r\n"
 | ε

<program> ::= <class> <newline>

<comment> ::= /* comment */

<newline> ::= "\r\n" <newline>
 | "\r\n"

<class> ::= public class <cname> {<newline> attribute}

<nested> ::= <class> <nested>
 | ε

<cname> ::= [A-Z][a-zA-Z0-9_]*

<attribute> ::= <amodifier> <variable>; "\r\n" <attribute>
 | <method>

<amodifier> ::= private
 | public
 | ε

<variable> ::= int <int>
 | char <char>
 | double <double>
 | boolean <boolean>
 | String <String>

<int> ::= <int_set>
 | <int_assign>

<int_set> ::= <id> , <int_set>
 | <id>

<id> ::= [a-zA-Z_][a-zA-Z0-9_]*

<int_assign> ::= <id> = <expression>, <int_assign>
 | <id> = <expression>

<char> ::= <char_set>
 | <char_assign>

```

<char_set> ::= <id> , <char_set>
            | <id>

<char_assign> ::= <id> = <CHAR> , <char_assign>
                | <id> = <CHAR>

<CHAR> ::= '[a-zA-Z0-9\n$|^]'

<double> ::= <double_set>
           | <double_assign>

<double_set> ::= <id> , <doubleset>
               | <id>

<double_assign> ::= <id> = <expression> , <double_assign>
                  | <id> = <expression>

<boolean> ::= <boolean_set>
           | <boolean_assign>

<boolean_set> ::= <id> , <boolean_set>
                | <id>

<boolean_assign> ::= <id> = <BOOLEAN> , <boolean_assign>
                  | <id> = <BOOLEAN>

<BOOLEAN> ::= true
            | false

<String> ::= <String_set>
           | <String_assign>

<String_set> ::= <id> , <String_set>
               | <id>

<String_assign> ::= <id> = <STRING> , <String_assign>
                  | <id> = <STRING>

<STRING> ::= ".*"

<method> ::= <mmodifier> <type> <id> ( <values> ) { <newline> <mattribute> <commands> }
<newline> <method>
           | <nested>

<mattribute> ::= <amodifier> <variable> ; <newline> <mattribute>
              | ε

<mmodifier> ::= public
              | private

```

```

<type>      ::= int
              | char
              | double
              | boolean
              | String
              | void

<values>     ::= <arg>
              | <arg2>
              | ε

<arg>        ::= int <id>, <arg>
              | char <id>, <arg>
              | double <id>, <arg>
              | boolean <id>, <arg>
              | String <id>, <arg>
              | <arg2>

<arg2>       ::= int <id>
              | char <id>
              | double <id>
              | boolean <id>
              | String <id>

<commands>   ::= <assign> <newline> <commands>
              | <object> <newline> <commands>
              | <access> <newline> <commands>
              | <do> <newline> <commands>
              | <for> <newline> <commands>
              | <if> <newline> <commands>
              | <switch> <newline> <commands>
              | <print> <newline> <commands>
              | <return> <newline> <commands>
              | <break> <newline> <commands>
              | ε

<object>     ::= <cname> <id> = new <cname>(<val>);

<val>        ::= <val1>
              | <val2>
              | ε

<val1>       ::= <INTEGER>, <val1>
              | <CHAR>, <val1>
              | <DOUBLE>, <val1>
              | <BOOLEAN>, <val1>
              | <STRING>, <val1>
              | <val2>

```

<INTEGER> ::= 0
 | [1-9][0-9]*

<DOUBLE> ::= (0|[1-9][0-9]*).[0-9]+d

<val2> ::= <INTEGER>
 | <CHAR>
 | <DOUBLE>
 | <BOOLEAN>
 | <STRING>

<access> ::= <id>.<id>;
 | <id>.<id>();

<assign> ::= <id> = <expression>;

<expression> ::= <literal>
 | <operation>
 | <id>.<id>()
 | new <cname>(<val>);
 | error "\r\n"

<operation> ::= <literal> <operator> <literal>
 | <operation> <operator> <operation>
 | <operation> <operator> <literal>
 | <literal> <operator> <operation>
 | (<operation>)
 | <literal>
 | error "\r\n"

<literal> ::= <val2>
 | <id>

<operator> ::= +
 | -
 | *
 | /

<do> ::= do { <newline> <commands>} <newline> while (<cond>);

<cond> ::= <log_expr1>
 | <log_expr2>

<log_expr1> ::= (<log_expr2>) <log_oper> <log_expr1>
 | (<log_expr2>)

<log_expr2> ::= <id> <oper> <second_operand>

<oper> ::= <comp_oper>
 | <log_oper>

<comp_oper> ::= >
 | <
 | ==
 | !=

<second_operand> ::= <INTEGER>
 | <DOUBLE>
 | <CHAR>
 | <BOOLEAN>
 | <id>

<log_oper> ::= &&
 | ||

<for> ::= for (<int_loop> <id> = <INTEGER>; <id> <comp_oper> <INTEGER>; <for_assign>) {
<newline> <commands> }

<int_loop> ::= int
 | ε

<for_assign> ::= <id> = <id> <operator> <INTEGER>

<if> ::= if (<cond>) { <newline> <commands> } <else_com>

<else_com> ::= <else_if> <else_com>
 | <newline> <else>
 | ε

<else_if> ::= else if (<cond>) { <newline> <commands> }

<else> ::= else { <newline> <commands> }

<switch> ::= switch (<operation>) { <newline> case <val2>: <newline> <commands> <case>
<default> }

<case> ::= case <val2>: <newline> <commands> <case>
 | ε

<default> ::= default: <newline> <commands>
 | ε

<print> ::= out.print(<STRING> <var_print>);

<var_print> ::= ,<id> <var_print>

| ϵ

<return> ::= return <operation>;

<break> ::= break;

<comment> ::= // .*

<mul_comment> ::= /* <com_message> */

<com_message> ::= .*
| .* <newline> .*

2.Παραδοχές

Κατά την συγγραφή της εργασίας θεωρήσαμε ότι εντός μιας κλάσης γίνεται πρώτα η δήλωση των μεταβλητών, στη συνέχεια η δήλωση των μεθόδων και τέλος η δήλωση των εμφωλευμένων κλάσεων. Στην συνέχεια κάναμε την παραδοχή ότι εντός μια μεθοδού δηλώνονται πρώτα οι μεταβλητές και μετά εκτελούνται οι εντολές. Επιπλέον εντός των μεθόδων δεν είναι δυνατή η δήλωση κάποιας νέας κλάσης ή μεθόδου. Σχετικά με την δήλωση πολλαπλών μεταβλητών θεωρήσαμε ότι σε μια γραμμή κώδικα δεν είναι δυνατή η ανάθεση τιμής σε μια μεταβλητή και η απλή δήλωση μια άλλης.

3.Τελικό Αρχείο FLEX

```
%{
    /* definitions */
    #include "parser1.tab.h"
}%

%option yylineno
%option noyywrap

%x mul_comment
%y sin_comment
INTEGER 0|[1-9][0-9]*
DOUBLE (0|[1-9][0-9]*)\.[0-9]+d
CHAR '[a-zA-Z0-9\\n\\$\\|\\^]'
NEWLINE \n
STRING \".+\"
CNAME [A-Z][a-zA-Z0-9]*
ID [a-zA-Z_][a-zA-Z0-9_]*
comment (\\t\\n)?.*

    /* rules */
%%

"int" {printf("int\\n"); return T_INT;}
"char" {printf("char\\n"); return T_CHAR;}
"double" {printf("double\\n"); return T_DOUBLE;}
"boolean" {printf("boolean\\n"); return T_BOOLEAN;}
"String" {printf("String\\n"); return T_STRING;}
"class" {printf("class\\n"); return T_CLASS;}
"new" {printf("new\\n"); return T_NEW;}
"return" {printf("return\\n"); return T_RETURN;}
"void" {printf("void\\n"); return T_VOID;}
"if" {printf("if\\n"); return T_IF;}
"else" {printf("else\\n"); return T_ELSE;}
"while" {printf("while\\n"); return T_WHILE;}
"do" {printf("do\\n"); return T_DO;}
"for" {printf("for\\n"); return T_FOR;}
"switch" {printf("switch\\n"); return T_SWITCH;}
"case" {printf("case\\n"); return T_CASE;}
"default" {printf("default\\n"); return T_DEFAULT;}
"break" {printf("break\\n"); return T_BREAK;}
"true" {printf("true\\n"); return T_TRUE;}
"false" {printf("false\\n"); return T_FALSE;}
"public" {printf("public\\n"); return T_PUBLIC;}
"private" {printf("private\\n"); return T_PRIVATE;}
"out" {printf("out\\n"); return T_OUT;}
"print" {printf("print\\n"); return T_PRINT;}
```

```
{INTEGER} {printf("Integer: %s\n", yytext); return T_INTEGER_VAL;}
{DOUBLE} {printf("Double: %s\n", yytext); return T_DOUBLE_VAL;}
{CHAR} {printf("Char: %s\n", yytext); return T_CHAR_VAL;}
{CNAME} {printf("Class Name: %s\n", yytext); return T_CNAME;}
{ID} {printf("Identifier: %s\n", yytext); return T_ID;}
{STRING} {printf("String: %s\n", yytext); return T_STRING_VAL;}
{NEWLINE} {printf("New Line\n"); return T_NEWLINE;}
```

```
"=" {printf("ASSIGN\n"); return T_ASSIGN;}
"+" {printf("PLUS\n"); return T_PLUS;}
"-" {printf("MINUS\n"); return T_MINUS;}
"*" {printf("MULTIPLY\n"); return T_MULTIPLY;}
"/" {printf("DIVIDE\n"); return T_DIVIDE;}
"%" {printf("MODULO\n"); return T_MODULO;}
">" {printf("GREATER\n"); return T_GREATER;}
"<" {printf("LESS\n"); return T_LESS;}
"==" {printf("EQUALS\n"); return T_EQUALS;}
"!=" {printf("NOT EQUALS\n"); return T_NEQUALS;}
"&&" {printf("AND\n"); return T_AND;}
"||" {printf("OR\n"); return T_OR;}
"(" {printf("LEFT PARENTHESIS\n"); return T_LPAR;}
")" {printf("RIGHT PARENTHESIS\n"); return T_RPAR;}
"{" {printf("LEFT BRACE\n"); return T_LBRACES;}
"}" {printf("RIGHT BRACE\n"); return T_RBRACES;}
"[" {printf("LEFT BRACKET\n"); return T_LBR;}
"]" {printf("RIGHT BRACKET\n"); return T_RBR;}
"." {printf("DOT\n"); return T_DOT;}
"," {printf("COMMA\n"); return T_COMMA;}
":" {printf("COLON\n"); return T_COLON;}
";" {printf("SEMICOLON\n"); return T_SEMICOLON;}
"\"" {printf("QUOTES\n"); return T_QUOTES;}
```

```
"/*" {BEGIN(mul_comment); printf("COMMENT START\n"); }
<mul_comment>"/" {BEGIN(INITIAL); printf("COMMENT END");}
<mul_comment>\n {return T_NEWLINE; }
<mul_comment>. {printf("%c", *yytext); return T_COMMENT;}
```

```
"//" {BEGIN(sin_comment); printf("COMMENT START\n"); }
<sin_comment> {BEGIN(INITIAL); printf("COMMENT END");}
<sin_comment>. {printf("%c", *yytext); return T_COMMENT;}
```

```
<<EOF>> {printf("End of File\n"); return T_EOF;}
```

```
[ \t] { }
```

```
. { /*ignore*/ }
```

```
%%
```

Ο τρόπος με τον οποίο υλοποιούμε τα comments είναι ο εξής: κάθε φορά που κατά την ανάλυση συναντάται `//` ή `/*` το flex τίθεται σε μία νέα κατάσταση `sin_comment` ή `mul_comment` αντίστοιχα εντός της οποίας δεν ισχύει κανένας κανόνας από τους προηγούμενους, ενώ μόλις συναντηθεί `newline` ή `*/` επιστρέφουμε στην αρχική κατάσταση του flex. Δεν καταφέραμε να υλοποιήσουμε τα σχόλια στο bison.

4.Τελικό Αρχείο BISON

```
%{
    /* definitions */
#include <stdio.h>

extern FILE *yyin;
extern int yylex();
extern int yylineno;
extern char *yytext;
int max_err = 0;

void yyerror(const char *message);
%}

#define parse.error verbose

    /* tokens */

%token T_INT
%token T_CHAR
%token T_DOUBLE
%token T_BOOLEAN
%token T_STRING
%token T_CLASS
%token T_NEW
%token T_RETURN
%token T_VOID
%token T_IF
%token T_ELSE
%token T_WHILE
%token T_DO
%token T_FOR
%token T_SWITCH
%token T_CASE
%token T_DEFAULT
%token T_BREAK
%token T_TRUE
%token T_FALSE
%token T_PUBLIC
%token T_PRIVATE
%token T_OUT
%token T_PRINT

%token T_INTEGER_VAL
```

```
%token T_DOUBLE_VAL
%token T_CHAR_VAL
%token T_CNAME
%token T_ID
%token T_STRING_VAL
%token T_NEWLINE "NEW LINE"
```

```
%token T_ASSIGN
%token T_PLUS
%token T_MINUS
%token T_MULTIPLY
%token T_DIVIDE
%token T_MODULO
%token T_GREATER
%token T_LESS
%token T_EQUALS
%token T_NEQUALS
%token T_AND
%token T_OR
%token T_LPAR
%token T_RPAR
%token T_LBRACES
%token T_RBRACES
%token T_LBR
%token T_RBR
%token T_DOT
%token T_COLON
%token T_SEMICOLON
%token T_COMMA
%token T_QUOTES
%token T_COMMENT
%token T_EOF 0
```

```
/* rules */
```

```
%%
start          : program start
                | error T_NEWLINE {yyerrok;}
                | %empty
                ;

program        : class newline
                ;
```

```

comment      : T_DIVIDE T_MULTIPLY T_COMMENT T_MULTIPLY T_DIVIDE
              ;

class        : T_PUBLIC T_CLASS cname T_LBRACES newline attribute T_RBRACES
              ;

nested       : class T_NEWLINE nested
              | %empty
              ;

cname        : T_CNAME
              ;

attribute    : amodifier variable T_SEMICOLON newline attribute
              | method
              ;

amodifier    : T_PRIVATE
              | T_PUBLIC
              | %empty
              ;

variable     : T_INT int
              | T_CHAR char
                | T_DOUBLE double
                | T_BOOLEAN boolean
              | T_STRING String
              ;

int          : int_set
              | int_assign
              ;

int_set      : id T_COMMA int_set
              | id
              ;

id           : T_ID
              ;

int_assign   : id T_ASSIGN expression T_COMMA int_assign
              | id T_ASSIGN expression
              ;

char         : char_set
              | char_assign
              ;

char_set     : id T_COMMA char_set

```

```

        | id
    ;

char_assign    : id T_ASSIGN T_CHAR_VAL T_COMMA char_assign
               | id T_ASSIGN T_CHAR_VAL
    ;

CHAR          : T_CHAR_VAL
    ;

double        : double_set
               | double_assign
    ;

double_assign  : id T_ASSIGN expression T_COMMA double_assign
               | id T_ASSIGN expression
    ;

double_set     : id T_COMMA double_set
               | id
    ;

boolean       : boolean_set
               | boolean_assign
    ;

boolean_set    : id T_COMMA boolean_set
               | id
    ;

boolean_assign : id T_ASSIGN BOOLEAN T_COMMA boolean_assign
               | id T_ASSIGN BOOLEAN
    ;

BOOLEAN       : T_TRUE
               | T_FALSE
    ;

String        : String_set
               | String_assign
    ;

String_set     : id T_COMMA String_set
               | id
    ;

String_assign  : id T_ASSIGN T_STRING_VAL T_COMMA String_assign
               | id T_ASSIGN T_STRING_VAL
    ;

```

STRING : T_STRING_VAL
;

method : mmodifier type id T_LPAR values T_RPAR T_LBRACES newline mattribute commands
T_RBRACES newline method
| nested
;

mattribute : amodifier variable T_SEMICOLON newline mattribute
| %empty
;

mmodifier : T_PUBLIC
| T_PRIVATE
;

type : T_INT
| T_CHAR
| T_DOUBLE
| T_BOOLEAN
| T_STRING
| T_VOID
;

values : arg
| arg2
| %empty
;

arg : T_INT id T_COMMA arg
| T_CHAR id T_COMMA arg
| T_DOUBLE id T_COMMA arg
| T_BOOLEAN id T_COMMA arg
| T_STRING id T_COMMA arg
| arg2
;

arg2 : T_INT id
| T_CHAR id
| T_DOUBLE id
| T_BOOLEAN id
| T_STRING id
;

commands : assign newline commands
| object newline commands
| access newline commands

```

| do newline commands
| for newline commands
| if newline commands
| switch newline commands
| print newline commands
| return newline commands
| break newline commands
| %empty
;

object      : cname id T_ASSIGN T_NEW cname T_LPAR val T_RPAR T_SEMICOLON
;

val         : val1
            | val2
            | %empty
;

val1        : INTEGER T_COMMA val1
            | CHAR T_COMMA val1
            | DOUBLE T_COMMA val1
            | BOOLEAN T_COMMA val1
            | STRING T_COMMA val1
            | val2
;

INTEGER     : T_INTEGER_VAL
;

DOUBLE      : T_DOUBLE_VAL
;

val2        : INTEGER
            | CHAR
            | DOUBLE
            | BOOLEAN
            | STRING
;

access      : id T_DOT id T_SEMICOLON
            | id T_DOT id T_LPAR T_RPAR T_SEMICOLON
;

assign      : id T_ASSIGN expression T_SEMICOLON
;

expression  : literal
            | id T_DOT id T_LPAR T_RPAR

```

```

        | T_NEW cname T_LPAR val T_RPAR
    | operation
        ;

newline      : T_NEWLINE newline
        | T_NEWLINE
        ;

operation    : literal operator literal
        | operation operator operation
        | operation operator literal
        | literal operator operation
        | T_LPAR operation T_RPAR
        | literal
        ;

literal      : val2
        | id
        ;

operator     : T_PLUS
        | T_MINUS
        | T_MULTIPLY
        | T_DIVIDE
        ;

do           : T_DO T_LBRACES newline commands T_RBRACES newline T_WHILE T_LPAR cond
T_RPAR T_SEMICOLON
        ;

cond         : log_expr1
        | log_expr2
        ;

log_expr1    : T_LPAR log_expr2 T_RPAR log_oper log_expr1
        | T_LPAR log_expr2 T_RPAR;

log_expr2    : id oper second_operand ;

oper         : comp_oper
        | log_oper;

comp_oper    : T_GREATER
        | T_LESS
        | T_EQUALS
        | T_NEQUALS ;

second_operand : INTEGER
        | DOUBLE

```

```

        | CHAR
        | BOOLEAN
        | id
        ;

log_oper      : T_AND
               | T_OR
               ;

for           : T_FOR T_LPAR int_loop id T_ASSIGN INTEGER T_SEMICOLON id comp_oper INTEGER
               T_SEMICOLON for_assign T_RPAR T_LBRACES newline commands T_RBRACES
               ;

int_loop      : T_INT
               | %empty
               ;

for_assign    : id T_ASSIGN id operator INTEGER
               ;

if           : T_IF T_LPAR cond T_RPAR T_LBRACES newline commands T_RBRACES else_com
               ;

else_com      : else_if else_com
               | newline else
               | %empty
               ;

else_if       : T_ELSE T_IF T_LPAR cond T_RPAR T_LBRACES newline commands T_RBRACES
               ;

else          : T_ELSE T_LBRACES newline commands T_RBRACES
               ;

switch        : T_SWITCH T_LPAR operation T_RPAR T_LBRACES newline T_CASE val2 T_COLON
               newline commands case default T_RBRACES
               ;

case          : T_CASE val2 T_COLON newline commands case
               | %empty
               ;

default       : T_DEFAULT T_COLON newline commands
               | %empty
               ;

print         : T_OUT T_DOT T_PRINT T_LPAR T_STRING_VAL var_print T_RPAR T_SEMICOLON
               ;

```

```

var_print      : T_COMMA id var_print
                | %empty
                ;

return         : T_RETURN operation T_SEMICOLON
                ;

break         : T_BREAK T_SEMICOLON
                ;

```

%%

```

int main(int argc, char *argv[]) {
    if (argc>1) {
        yyin = fopen(argv[1], "r");
        if (yyin==NULL) {
            perror("Error opening file");
            return -1;
        }
    }
    yyparse();

    fclose(yyin);
    if(max_err>=1)
        printf("Unsuccessful parsing, total errors: %d", max_err);
    else if(max_err==0)
        printf("Successful parsing, no errors detected");
    return 0;
}

void yyerror(const char *message) {
    max_err ++;
    printf("Error at line %d caused by %s : %s\n", yylineno, yytext, message);
}

```

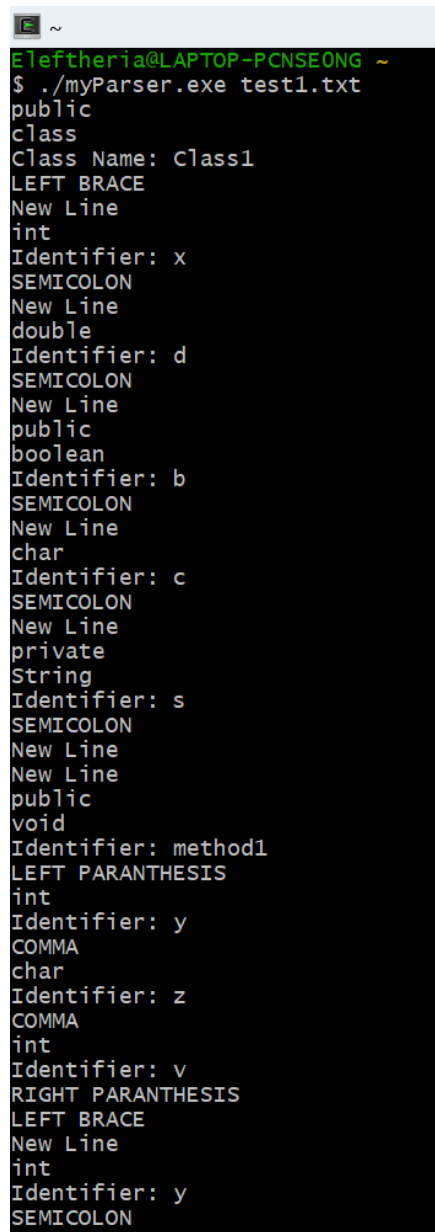
Δεν καταφέραμε να υλοποιήσουμε σωστά το 4ο ερώτημα, καθώς παρόλο που η ανάλυση δε σταματά στη πρώτη εμφάνιση λάθους εμφανίζεται ειδοποίηση λάθους σε

κάθε γραμμή μετά τη πρώτη εμφάνιση λάθους. Επίσης, ο τρόπος με τον οποίο υλοποιήσαμε τον συντακτικό αναλυτή δεν επιτρέπει η 1η μέθοδος που θα δηλωθεί να έχει άλλο τύπο επιστροφής εκτός από το void.

5. Παραδείγματα Εκτέλεσης

Ερώτημα 1:

- Καλούμε το πρόγραμμα του αρχείου test1.txt, το οποίο αποτελείται από δύο κλάσεις με δήλωση μεταβλητών και μεθόδων. Οι μέθοδοι περιέχουν δήλωση μεταβλητών και εντολές ανάθεσης, βρόγχου, ελέγχου, εκτύπωσης στην οθόνη, επιστροφής από μέθοδο, τερματισμού βρόγχου, καθώς και δημιουργίας αντικειμένου κλάσης και πρόσβαση σε μέλος αυτής.



```
Eleftheria@LAPTOP-PCNSE0NG ~  
$ ./myParser.exe test1.txt  
public  
class  
Class Name: class1  
LEFT BRACE  
New Line  
int  
Identifier: x  
SEMICOLON  
New Line  
double  
Identifier: d  
SEMICOLON  
New Line  
public  
boolean  
Identifier: b  
SEMICOLON  
New Line  
char  
Identifier: c  
SEMICOLON  
New Line  
private  
String  
Identifier: s  
SEMICOLON  
New Line  
New Line  
public  
void  
Identifier: method1  
LEFT PARENTHESIS  
int  
Identifier: y  
COMMA  
char  
Identifier: z  
COMMA  
int  
Identifier: v  
RIGHT PARENTHESIS  
LEFT BRACE  
New Line  
int  
Identifier: y  
SEMICOLON
```

```
~  
New Line  
int  
Identifier: v  
SEMICOLON  
New Line  
int  
Identifier: w  
SEMICOLON  
New Line  
char  
Identifier: z  
SEMICOLON  
New Line  
Identifier: y  
ASSIGN  
Identifier: v  
PLUS  
Integer: 1  
SEMICOLON  
New Line  
New Line  
do  
LEFT BRACE  
New Line  
Identifier: w  
ASSIGN  
Identifier: w  
PLUS  
Integer: 1  
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
while  
LEFT PARENTHESIS  
Identifier: w  
GREATER  
Integer: 10  
RIGHT PARENTHESIS  
SEMICOLON  
New Line  
New Line  
for  
LEFT PARENTHESIS  
int  
Identifier: i  
ASSIGN  
Integer: 0
```



```
SEMICOLON  
Identifier: i  
LESS  
Integer: 10  
SEMICOLON  
Identifier: i  
ASSIGN  
Identifier: i  
PLUS  
Integer: 1  
RIGHT PARANTHESIS  
LEFT BRACE  
New Line  
out  
DOT  
print  
LEFT PARANTHESIS  
String: "i="  
COMMA  
Identifier: i  
RIGHT PARANTHESIS  
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
New Line  
if  
LEFT PARANTHESIS  
Identifier: y  
EQUALS  
Integer: 1  
RIGHT PARANTHESIS  
LEFT BRACE  
New Line  
Identifier: v  
ASSIGN  
Identifier: v  
PLUS  
Integer: 2  
SEMICOLON  
New Line  
out  
DOT  
print  
LEFT PARANTHESIS  
String: "v="  
COMMA  
Identifier: v
```

 ~
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
else
if
LEFT PARANTHESIS
Identifier: y
EQUALS
Integer: 2
RIGHT PARANTHESIS
LEFT BRACE
New Line
Identifier: v
ASSIGN
Identifier: v
PLUS
Integer: 3
SEMICOLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "v="
COMMA
Identifier: v
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
New Line
else
LEFT BRACE
New Line
Identifier: v
ASSIGN
Identifier: v
PLUS
Integer: 4
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
switch
LEFT PARANTHESIS
Identifier: z

 ~
RIGHT PARANTHESIS
LEFT BRACE
New Line
case
Char: 'a'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "a"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
case
Char: 'b'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "b"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
case
Char: 'c'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "c"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line

 ~
default
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "Wrong choice!"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
Class Name: Class2
Identifier: obj
ASSIGN
new
Class Name: Class2
LEFT PARANTHESIS
RIGHT PARANTHESIS
SEMICOLON
New Line
Identifier: obj
DOT
Identifier: string
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
public
String
Identifier: method2
LEFT PARANTHESIS
RIGHT PARANTHESIS
LEFT BRACE
New Line
String
Identifier: str
SEMICOLON
New Line
return
Identifier: str
SEMICOLON

```
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
RIGHT BRACE  
New Line  
New Line  
public  
class  
Class Name: Class2  
LEFT BRACE  
New Line  
boolean  
Identifier: bool  
SEMICOLON  
New Line  
String  
Identifier: string  
SEMICOLON  
New Line  
int  
Identifier: j  
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
End of File  
Successful parsing, no errors detected  
Eleftheria@LAPTOP-PCNSEONG ~
```

Παρατηρούμε ότι εκτυπώνεται μήνυμα επιτυχούς ανάλυσης, συνεπώς το πρόγραμμα του αρχείου test1.txt είναι ορθώς γραμμένο.

- Καλούμε το πρόγραμμα του αρχείου test2.txt, το οποίο αποτελείται από δύο κλάσεις, εμφωλευμένη η μία κλάση μέσα στην άλλη. Η κάθε κλάση περιέχει δήλωση μεταβλητών και μεθόδων.

```
Eleftheria@LAPTOP-PCNSEONG ~  
$ ./myParser.exe test2.txt  
public  
class  
Class Name: ClassA  
LEFT BRACE  
New Line  
int  
Identifier: a  
SEMICOLON  
New Line  
New Line  
private  
void  
Identifier: methodA  
LEFT PARANTHESIS  
int  
Identifier: x  
RIGHT PARANTHESIS  
LEFT BRACE  
New Line  
out  
DOT  
print  
LEFT PARANTHESIS  
String: "x="  
COMMA  
Identifier: x  
RIGHT PARANTHESIS  
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
New Line  
public  
class  
Class Name: ClassB  
LEFT BRACE  
New Line  
double  
Identifier: b  
SEMICOLON  
New Line  
New Line  
public  
void  
Identifier: methodB  
LEFT PARANTHESIS  
  
RIGHT PARANTHESIS  
LEFT BRACE  
New Line  
out  
DOT  
print  
LEFT PARANTHESIS  
String: "MethodB"  
RIGHT PARANTHESIS  
SEMICOLON  
New Line  
RIGHT BRACE  
New Line  
RIGHT BRACE  
New Line  
RIGHT BRACE  
New Line  
End of File  
Successful parsing, no errors detected  
Eleftheria@LAPTOP-PCNSEONG ~
```

Παρατηρούμε ότι εκτυπώνεται μήνυμα επιτυχούς ανάλυσης, συνεπώς το πρόγραμμα του αρχείου test2.txt είναι ορθώς γραμμένο.

- Καλούμε το πρόγραμμα του αρχείου test3.txt, το οποίο αποτελείται από το πρόγραμμα του αρχείου test1.txt όμως με ένα λάθος. Συγκεκριμένα, στην γραμμή 5 όπου γίνεται δήλωση μίας μεταβλητής char λείπει το id της μεταβλητής.

```
Eleftheria@LAPTOP-PCNSE0NG ~  
$ ./myParser.exe test3.txt  
public  
class  
Class Name: c1ass1  
LEFT BRACE  
New Line  
int  
Identifier: x  
SEMICOLON  
New Line  
double  
Identifier: d  
SEMICOLON  
New Line  
public  
boolean  
Identifier: b  
SEMICOLON  
New Line  
char  
SEMICOLON  
Error at line 5 caused by ; : syntax error, unexpected T_SEMICOLON, expecting T_ID  
Unsuccessful parsing, total errors: 1  
Eleftheria@LAPTOP-PCNSE0NG ~
```

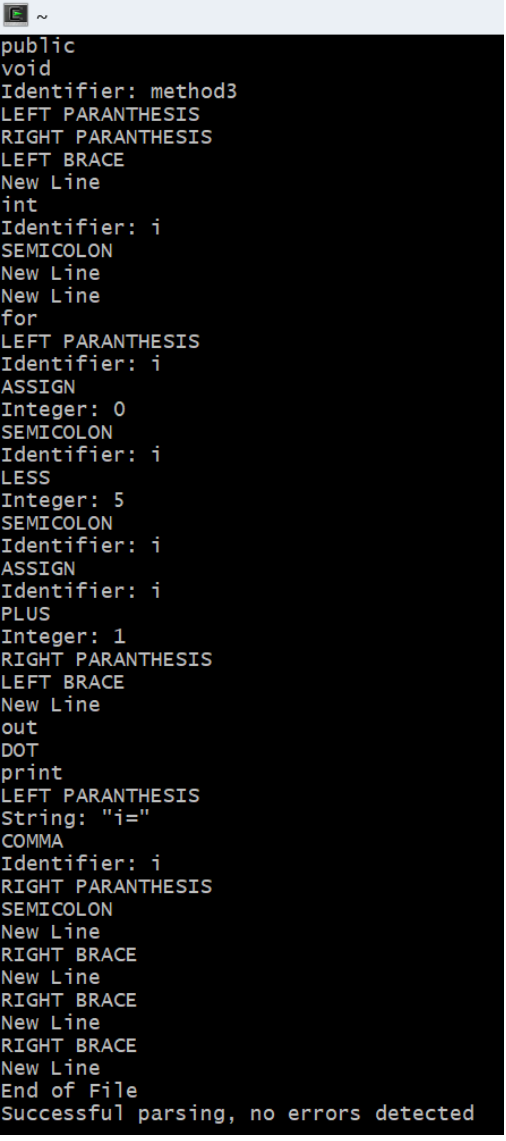
Παρατηρούμε ότι εκτυπώνεται στην οθόνη ο αριθμός της γραμμής του σφάλματος, ο λόγος που προκύπτει αυτό και μήνυμα ανεπιτυχούς ανάλυσης, ενώ η ανάλυση διακόπτεται εκεί.

Ερώτημα 2:

- Καλούμε το πρόγραμμα του αρχείου test4.txt, το οποίο αποτελείται από μία κλάση με δήλωση μεταβλητών και μεθόδων. Κατά την δήλωση μίας μεταβλητής γίνεται ανάθεση τιμής, ενώ γίνεται και δήλωση πολλαπλών μεταβλητών ίδιου τύπου σε μια γραμμή.

```
Eleftheria@LAPTOP-PCNSE0NG ~  
$ ./myParser.exe test4.txt  
public  
class  
Class Name: Class3  
LEFT BRACE  
New Line  
int  
Identifier: x  
ASSIGN  
Integer: 0  
COMMA  
Identifier: y  
ASSIGN  
Integer: 1  
COMMA  
Identifier: z  
ASSIGN  
Integer: 3  
SEMICOLON  
New Line  
public  
int  
Identifier: w  
COMMA  
Identifier: v  
SEMICOLON  
New Line  
double  
Identifier: a  
ASSIGN  
Double: 3.14d  
COMMA  
Identifier: b  
ASSIGN  
Double: 54.65d  
COMMA  
Identifier: c  
ASSIGN  
Double: 5322.67d  
SEMICOLON  
New Line  
double  
Identifier: d  
COMMA  
Identifier: e  
SEMICOLON  
New Line
```

```
boolean
Identifier: f
ASSIGN
false
COMMA
Identifier: t
ASSIGN
true
SEMICOLON
New Line
boolean
Identifier: g
COMMA
Identifier: h
COMMA
Identifier: j
SEMICOLON
New Line
char
Identifier: attr1
ASSIGN
Char: 'a'
COMMA
Identifier: attr2
ASSIGN
Char: '2'
SEMICOLON
New Line
private
char
Identifier: k
COMMA
Identifier: l
SEMICOLON
New Line
String
Identifier: s
ASSIGN
String: "attribute",str="string"
SEMICOLON
New Line
String
Identifier: m
COMMA
Identifier: n
SEMICOLON
New Line
New Line
```



```
public
void
Identifier: method3
LEFT PARANTHESIS
RIGHT PARANTHESIS
LEFT BRACE
New Line
int
Identifier: i
SEMICOLON
New Line
New Line
for
LEFT PARANTHESIS
Identifier: i
ASSIGN
Integer: 0
SEMICOLON
Identifier: i
LESS
Integer: 5
SEMICOLON
Identifier: i
ASSIGN
Identifier: i
PLUS
Integer: 1
RIGHT PARANTHESIS
LEFT BRACE
New Line
out
DOT
print
LEFT PARANTHESIS
String: "i="
COMMA
Identifier: i
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
New Line
RIGHT BRACE
New Line
RIGHT BRACE
New Line
RIGHT BRACE
New Line
End of File
Successful parsing, no errors detected
```

Παρατηρούμε ότι εκτυπώνεται μήνυμα επιτυχούς ανάλυσης, συνεπώς το πρόγραμμα του αρχείου test4.txt είναι ορθώς γραμμένο.

Ερώτημα 4:

- Καλούμε το πρόγραμμα του αρχείου test5.txt, το οποίο αποτελείται από το πρόγραμμα του αρχείου test1.txt όμως με ένα λάθος. Συγκεκριμένα, στην γραμμή 61 όπου γίνεται δήλωση μίας μεταβλητής boolean λείπει το id της μεταβλητής. Σε αυτή την περίπτωση μετά το λάθος η ανάλυση δεν πρέπει να σταμάτα, αλλά να συνεχίζει κανονικά.

```
Eleftheria@LAPTOP-PCNSE0NG ~  
$ ./myParser.exe test5.txt  
public  
class  
Class Name: Class1  
LEFT BRACE  
New Line  
int  
Identifier: x  
SEMICOLON  
New Line  
double  
Identifier: d  
SEMICOLON  
New Line  
public  
boolean  
Identifier: b  
SEMICOLON  
New Line  
char  
Identifier: c  
SEMICOLON  
New Line  
private  
String  
Identifier: s  
SEMICOLON  
New Line  
New Line  
public  
void  
Identifier: method1  
LEFT PARENTHESIS  
int  
Identifier: y  
COMMA  
char  
Identifier: z  
COMMA  
int  
Identifier: v  
RIGHT PARENTHESIS  
LEFT BRACE  
New Line  
int  
Identifier: y  
SEMICOLON
```

```
~
New Line
int
Identifier: v
SEMICOLON
New Line
int
Identifier: w
SEMICOLON
New Line
char
Identifier: z
SEMICOLON
New Line
Identifier: y
ASSIGN
Identifier: v
PLUS
Integer: 1
SEMICOLON
New Line
New Line
do
LEFT BRACE
New Line
Identifier: w
ASSIGN
Identifier: w
PLUS
Integer: 1
SEMICOLON
New Line
RIGHT BRACE
New Line
while
LEFT PARENTHESIS
Identifier: w
GREATER
Integer: 10
RIGHT PARENTHESIS
SEMICOLON
New Line
New Line
for
LEFT PARENTHESIS
int
Identifier: i
ASSIGN
Integer: 0
```

```
SEMICOLON
Identifier: i
LESS
Integer: 10
SEMICOLON
Identifier: i
ASSIGN
Identifier: i
PLUS
Integer: 1
RIGHT PARANTHESIS
LEFT BRACE
New Line
out
DOT
print
LEFT PARANTHESIS
String: "i="
COMMA
Identifier: i
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
if
LEFT PARANTHESIS
Identifier: y
EQUALS
Integer: 1
RIGHT PARANTHESIS
LEFT BRACE
New Line
Identifier: v
ASSIGN
Identifier: v
PLUS
Integer: 2
SEMICOLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "v="
COMMA
Identifier: v
```

```
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
else
if
LEFT PARANTHESIS
Identifier: y
EQUALS
Integer: 2
RIGHT PARANTHESIS
LEFT BRACE
New Line
Identifier: v
ASSIGN
Identifier: v
PLUS
Integer: 3
SEMICOLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "v="
COMMA
Identifier: v
RIGHT PARANTHESIS
SEMICOLON
New Line
RIGHT BRACE
New Line
else
LEFT BRACE
New Line
Identifier: v
ASSIGN
Identifier: v
PLUS
Integer: 4
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
switch
LEFT PARANTHESIS
Identifier: z
```

 ~
RIGHT PARANTHESIS
LEFT BRACE
New Line
case
Char: 'a'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "a"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
case
Char: 'b'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "b"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
case
Char: 'c'
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "c"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line

```
default
COLON
New Line
out
DOT
print
LEFT PARANTHESIS
String: "Wrong choice!"
RIGHT PARANTHESIS
SEMICOLON
New Line
break
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
Class Name: Class2
Identifier: obj
ASSIGN
new
Class Name: Class2
LEFT PARANTHESIS
RIGHT PARANTHESIS
SEMICOLON
New Line
Identifier: obj
DOT
Identifier: string
SEMICOLON
New Line
RIGHT BRACE
New Line
New Line
public
String
Identifier: method2
LEFT PARANTHESIS
RIGHT PARANTHESIS
LEFT BRACE
New Line
String
Identifier: str
SEMICOLON
New Line
return
Identifier: str
SEMICOLON
```

```
New Line
RIGHT BRACE
New Line
RIGHT BRACE
New Line
New Line
public
class
Class Name: Class2
LEFT BRACE
New Line
boolean
SEMICOLON
Error at line 61 caused by ; : syntax error, unexpected T_SEMICOLON, expecting T_ID
New Line
String
Error at line 62 caused by String : syntax error, unexpected T_STRING, expecting T_EOF
Identifier: string
SEMICOLON
New Line
int
Error at line 63 caused by int : syntax error, unexpected T_INT, expecting T_EOF
Identifier: j
SEMICOLON
New Line
RIGHT BRACE
Error at line 64 caused by } : syntax error, unexpected T_RBACES, expecting T_EOF
New Line
End of File
Unsuccessful parsing, total errors: 4
Eleftheria@LAPTOP-PCN5EQNG ~
```

Παρατηρούμε ότι εκτυπώνεται στην οθόνη ο αριθμός της γραμμής του σφάλματος και ο λόγος που προκύπτει αυτό, ενώ η ανάλυση συνεχίζει.

6.Παραδοτέα Αρχεία και Makefile

1. `lexer.l`: αρχείο flex που αντιστοιχεί στα 2 πρώτα ερωτήματα. Περιέχει το λεξικό αναλυτή της εργασίας και έχει γίνει σε αυτό `include` το headerfile `parser.tab.h`.
2. `lexer1.l`: τελικό αρχείο λεξικού αναλυτή που αντιστοιχεί και στο 4ο ερώτημα. Στο συγκεκριμένο γίνεται `include` το headerfile `parser1.tab.h`
3. `parser.y`: αρχείο του συντακτικού αναλυτή που αντιστοιχεί στα 2 πρώτα ερωτήματα της εργασίας.
4. `parser1.y`: τελικό αρχείο συντακτικού αναλυτή που περιέχει και το 4ο ερώτημα της εργασίας.
5. `lex.yy.c`: πρόγραμμα του λεκτικού αναλυτή. Παράγεται αυτόματα από το flex.
6. `parser1.tab.c`: παραγόμενο αρχείο του bison. Είναι το αρχείο που περιέχει το source code για τη συντακτική ανάλυση.
7. `parser1.tab.h`: header file του bison compiler. Παράγεται αυτόματα από το bison.
8. `myParser.exe`: εκτελέσιμο αρχείο που παράγεται.
9. `test1.txt`: δοκιμαστικό αρχείο που αφορά το 1ο ερώτημα. Είναι ορθώς γραμμένο.
10. `test2.txt`: δοκιμαστικό αρχείο που αφορά το 1ο ερώτημα. Αποτελεί παράδειγμα εμφωλευμένης κλάσης. Είναι ορθώς γραμμένο.
11. `test3.txt`: δοκιμαστικό αρχείο που αφορά το 1ο ερώτημα. Περιέχει λάθος για να επιδείξουμε τη διαδικασία εμφάνισης σφαλμάτων του προγράμματος.
12. `test4.txt`: δοκιμαστικό αρχείο που αφορά το 2ο ερώτημα. Συγκεκριμένα περιέχει παραδείγματα πολλαπλής δήλωσης μεταβλητών σε μία γραμμή και πολλαπλής ανάθεσης τιμών σε μία γραμμή.
13. `test5.txt`: δοκιμαστικό αρχείο που αφορά το 4ο ερώτημα. Συγκεκριμένα παρουσιάζει τη λειτουργία του προγράμματος να μη τερματίζει με την εμφάνιση σφάλματος, αλλά να συνεχίζει την ανάλυση.

Με τις ακόλουθες εντολές δημιουργούμε τα κατάλληλα αρχεία και εκτελούμε την ανάλυση:

- `flex lexer.l` (ή `flex lexer1.l`)
- `bison -d -t parser.y` (ή `bison -d -t parser1.y`)
- `gcc lex.yy.c parser.tab.c -o myParser` (ή `gcc lex.yy.c parser1.tab.c -o myParser`)
- `./myParser.exe testn.txt` (όπου `n` ο αριθμός του δοκιμαστικού αρχείου που θέλουμε να τρέξουμε)